

Collateral Health — Take-Home Exercise

Before you start

This should take **no more than 3 hours**. It is deliberately scoped so that a strong solution fits comfortably in that window. If you hit a wall, **stop and write down what you would have done next** — we would much rather see clear thinking about the remaining work than a rushed, complete-but-muddy submission.

We are not evaluating how much you can build. We are evaluating **how you think**: how you model a domain, how you decide what to build and what to leave out, how you test, and how you communicate your reasoning. There is no "gotcha," and there is no single correct architecture.

On incomplete or contradictory rules. The rules below are close to a real domain, and like most real specs they have gaps and a few places where the literal wording and the likely intent may diverge. Where you find a rule that is incomplete, ambiguous, or contradictory, do not get stuck — **make a reasonable decision, write down the assumption you made and why, and keep going.**

What to deliver

Think of the deliverable as code that we would insert into our system and hook it up so that it can be consumed by HTTP controllers and background jobs that need to compute the health of a Collateral Arrangement (CA).

Do add a README please, covering: the assumptions you made, the key design decisions and tradeoffs, and anything you deliberately left out or would do differently with more time.

Use whatever language and tools you are most fluent in. (For reference, we are a TypeScript shop).

The domain

Glossary

- **Collateral Arrangement (CA)**: the thing whose health we are computing. It holds a balance of collateral and backs a loan.
- **Collateral balance**: an amount of some asset held in the CA (e.g. an amount of BTC).

- **Requirement:** the amount of the loan the CA is backing, denominated in some asset (e.g. an amount of USDC). This is the value the collateral must cover.
- **LTV (loan-to-value):** a percentage (e.g. 80%). A CA has three of them. -
- Limit:** a monetary threshold derived from the collateral and an LTV. -
- Status:** the health of the CA — one of the five statuses below.

Setup

A CA holds a **collateral balance** in one asset and has a **requirement** (the loan amount) denominated in **another asset**. For example, a CA might hold a balance in **BTC** and have a requirement in **USDC**. You may mock any prices you need.

A CA has three LTVs (Initial LTV, Maintenance LTV and Liquidation LTV), where:

None

$$\text{Initial} \leq \text{Maintenance} \leq \text{Liquidation}$$

Each LTV produces a monetary **limit**:

None

$$\text{limit} = \text{collateral value} \times \text{LTV}$$

The five statuses

- **Good Standing**
- **Near Margin**
- **Initial Margin Call**
- **Maintenance Margin Call**
- **Liquidation**

Base rules — classifying by where the requirement lands

Comparing the **requirement** against the limits:

1. requirement **below** the Initial limit → **Good Standing**
2. requirement **at or above** Initial and **below** Maintenance → **Near Margin**
3. requirement **at or above** Maintenance and **below** Liquidation → **Maintenance Margin Call**
4. requirement **at or above** Liquidation → **Liquidation**

Precedence. The three LTVs are allowed to be equal to one another. When a requirement lands exactly on a shared threshold, the **more severe** status wins: Liquidation takes precedence over Maintenance, and Maintenance over Initial.

Events — what triggered the recompute

A status is (re)computed in response to an **event**. There are two kinds:

- A **link** event — the loan being linked to the CA.
- Any **other** recompute — e.g. a price move, a change in balance, a repayment.

Rules that depend on the event and the previous status

5. **On a link event:** the only statuses a link may *produce* are **Good Standing** or **Initial Margin Call**. If the requirement lands at or above the Initial limit, the status becomes **Initial Margin Call** (regardless of which higher limit is crossed); otherwise it is **Good Standing**.

- **However, a link event must never override a preexisting Maintenance Margin Call or Liquidation.** If the CA is already in one of those two statuses, a link leaves it unchanged.

6. A CA in **Initial Margin Call** cannot be promoted to Maintenance Margin Call or Liquidation.

7. A CA in **Maintenance Margin Call** or **Liquidation** does not return to Good Standing until the requirement lands below the Initial limit.

8. A **Maintenance Margin Call** can be promoted to **Liquidation**.

Worked example

A single concrete example, so the mechanics are unambiguous:

Input	Value
Collateral balance	2 BTC
Mock price	30,000 USDC/BTC
Collateral value	60,000 USDC

Input	Value
-------	-------

Initial LTV	50%
Maintenance LTV	65%
Liquidation LTV	80%

Derived limits:

Limit	Calculation	Value
Initial limit	$60,000 \times 50\%$	30,000 USDC
Maintenance limit	$60,000 \times 65\%$	39,000 USDC
Liquidation limit	$60,000 \times 80\%$	48,000 USDC

Now suppose the **requirement is 42,000 USDC**. It lands at or above the Maintenance limit (39,000) and below the Liquidation limit (48,000), so — on an ordinary (non-link) recompute — the status is **Maintenance Margin Call**.

If that same CA had *just been linked* instead: the requirement is at or above the Initial limit, so the status would be **Initial Margin Call**.

A note on scope

You do **not** need persistence, a UI, a CLI, HTTP, or any infrastructure. A well-modelled library with a clear public API is exactly what we are looking for. Spend your time on the domain logic and on making your reasoning legible to consumers of your solution.

Good luck — and remember, if something is unclear, decide, document it, and move on.